

Documentation for the incompressible Navier-Stokes Matlab codes

William D. Henshaw^{a,1,}

^a*Department of Mathematical Sciences, Rensselaer Polytechnic Institute, Troy, NY 12180, USA*

Abstract

Here is documentation and notes for the incompressible Navier-Stokes Matlab codes.

Contents

1	Matlab files	2
2	Introduction, governing equations, boundary conditions	2
2.1	Boundary conditions	2
3	Numerical approach	5
3.1	Discrete approximations in space	5
3.2	Time-stepping algorithms	5
3.3	Free surface evolution	5
4	Numerical Results	5
4.1	Manufactured solutions	5
4.2	Taylor Green vortex	7
4.3	Shear flow in a periodic channel	8

Email address: `henshw@rpi.edu` (William D. Henshaw)

1. Matlab files

ins.m	Main routine .
runChecks.m	Run regression tests.
runGridConvergence.m	Run a grid convergence test.
advanceAdams.m	Adams time-stepping.
advanceIM.m	IMEX time-stepping.
advancePC.m	Predictor-Corrector time-stepping.
applyBoundaryConditions.m	apply (explicit) boundary conditions
pressureEquation.m	form pressure equation and solve
evalMap.m	eval the grid mapping and derivatives
formImplicitTimeSteppingMatrix.m	form systems for implicit time-stepping
solveImplicitTimeStep.m	solve the implicit time-step equations
getInitialConditions.m	assign initial conditions
getUt.m	evaluate momentum equations
getTimeStep.m	determine the time-step Δt
plotSolution.m	plot solution contours and streamlines
plotStreamLines.m	stream line plotter

Table 1: Selected ins Matlab files.

2. Introduction, governing equations, boundary conditions

The `ins.m` Matlab code solves the incompressible Navier-Stokes equations in velocity-pressure form,

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} + \frac{1}{\rho} \nabla p = \nu \Delta \mathbf{u} + \mathbf{g} + \mathbf{f}(\mathbf{x}, t), \quad \mathbf{x} \in \Omega, \quad t > 0, \quad (1a)$$

$$\frac{1}{\rho} \Delta p = -\nabla \mathbf{u} : \nabla \mathbf{u} + \nabla \cdot \mathbf{f}, \quad \mathbf{x} \in \Omega, \quad t > 0, \quad (1b)$$

$$\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}) \quad \mathbf{x} \in \Omega, \quad (1c)$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, 0) = 0 \quad \mathbf{x} \in \partial\Omega, \quad (1d)$$

where $\nu = \mu/\rho$ is the kinematic viscosity coefficient, $\mathbf{g}$ is the gravity vector and $\mathbf{f}(\mathbf{x}, t)$ is a body forcing. These equations are augmented with appropriate boundary conditions as discussed below.

For problems with surface tension, the traction free boundary condition for the pressure is

$$-p = \mathbf{n}^T \boldsymbol{\tau} \mathbf{n} = \gamma \kappa \quad (2)$$

where $\tau_{ij} = \mu(\partial_{x_j} u_i + \partial_{x_i} u_j)$, γ is the coefficient of surface tension and κ is the curvature. Non-dimensional numbers associated with the surface tension are

$$\text{We} = \frac{\rho v^2 L}{\gamma}, \quad \text{Weber number: ratio of inertial to surface tension forces,} \quad (3a)$$

$$\text{Ca} = \frac{\mu v}{\gamma} \quad \text{Capillary number: ratio of viscous to surface tension forces,} \quad (3b)$$

$$\text{Bo} = \frac{\Delta \rho g L}{\gamma} \quad \text{Bond number: ratio of buoyancy to surface tension forces,} \quad (3c)$$

$$\text{Oh} = \frac{\mu}{\sqrt{\rho \gamma L}} \quad \text{Ohnesorge number: relates viscous to inertial and surface tension forces,} \quad (3d)$$

where v is the velocity scale, L the length scale, and where $\Delta \rho$ is the density difference between the two fluids (or ρ for a free surface).

2.1. Boundary conditions

Let $\mathbf{n}(\mathbf{x})$ denote the outward unit normal vector to a point $\mathbf{x}$ on a boundary. Let $\mathbf{t}$ denote the corresponding unit tangent vector. The stress tensor is $\boldsymbol{\sigma} = -p\mathbf{I} + \boldsymbol{\tau}$ with $\boldsymbol{\tau} = \mu(\nabla \mathbf{u} + (\nabla \mathbf{u})^T)$. They have components

$$\sigma_{ij} = -p\delta_{ij} + \mu(\partial_{x_j} u_i + \partial_{x_i} u_j), \quad (4a)$$

$$\tau_{ij} = \mu(\partial_{x_j} u_i + \partial_{x_i} u_j). \quad (4b)$$

In two dimensions,

$$\boldsymbol{\sigma} = \begin{bmatrix} -p + 2\mu u_x & \mu(u_y + v_x) \\ \mu(u_y + v_x) & -p + 2\mu v_y \end{bmatrix} \quad (5)$$

Here are the physical boundary conditions. Numerically, additional compatibility conditions will be used.

No-slip wall or inflow.

$$\mathbf{u}(\mathbf{x}, t) = \mathbf{g}(\mathbf{x}, t), \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (6a)$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, t) = 0, \quad \mathbf{x} \in \partial\Omega, \quad t > 0. \quad (6b)$$

Slip wall.

$$\mathbf{n}^T \mathbf{u}(\mathbf{x}, t) = 0 \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (7a)$$

$$\mathbf{t}^T \boldsymbol{\sigma} \mathbf{n} = 0, \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (7b)$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, t) = 0 \quad \mathbf{x} \in \partial\Omega, \quad t > 0. \quad (7c)$$

With explicit time-stepping, the slipwall condition (7a) can be imposed by first updating the values on the boundary using the interior PDE and then using (7a) to set the normal component to zero,

$$\mathbf{u} \leftarrow \mathbf{u} - (\mathbf{n}^T \mathbf{u}) \mathbf{n}. \quad (8)$$

With implicit time-stepping, the slipwall condition (7a) can be combined with the interior PDE to give the two equations (thought of as determining the boundary points)

$$\mathbf{n} \mathbf{n}^T \mathbf{u}(\mathbf{x}, t) + (I - \mathbf{n} \mathbf{n}^T)(\text{PDE}) = 0. \quad (9)$$

The slipWall conditions (7b) and (7c) can be written as (see comments below for traction BC)

$$(\nabla \cdot \mathbf{u}(\mathbf{x}, t)) \mathbf{n} + (I - \mathbf{n} \mathbf{n}^T) \boldsymbol{\sigma} \mathbf{n} = 0. \quad (10)$$

which re thought of as determining the ghost points.

Outflow.

$$a_0 p(\mathbf{x}, t) + a_1 \partial_n p(\mathbf{x}, t) = g(\mathbf{x}, t), \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (11a)$$

$$\partial_n^2 (\mathbf{t} \cdot \mathbf{u}) \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (11b)$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, t) = 0 \quad \mathbf{x} \in \partial\Omega, \quad t > 0. \quad (11c)$$

Pressure inflow.

$$p(\mathbf{x}, t) = g_1(\mathbf{x}, t), \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (12a)$$

$$\mathbf{t} \cdot \mathbf{u} = g_2 \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (12b)$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, t) = 0 \quad \mathbf{x} \in \partial\Omega, \quad t > 0. \quad (12c)$$

Traction (or free surface). The traction (or free surface) boundary condition is

$$\boldsymbol{\sigma} \mathbf{n} = \gamma \kappa(\mathbf{x}) \mathbf{n}, \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (13a)$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, t) = 0 \quad \mathbf{x} \in \partial\Omega, \quad t > 0. \quad (13b)$$

where γ is the coefficient of surface tension and κ is the curvature. For a surface in 3D parameterized as $\mathbf{y}(r, s)$, the curvature is

$$\kappa = \frac{\left(\|\mathbf{y}_r\|^2 \mathbf{y}_{ss} - 2(\mathbf{y}_r \cdot \mathbf{y}_s) \mathbf{y}_{rs} + \|\mathbf{y}_s\|^2 \mathbf{y}_{rr} \right) \cdot \mathbf{n}}{\|\mathbf{y}_r\|^2 \|\mathbf{y}_s\|^2 - (\mathbf{y}_r \cdot \mathbf{y}_s)^2}. \quad (13c)$$

For a curve $\mathbf{y}(r)$ in 2D this reduces to

$$\kappa = \frac{\mathbf{y}_{rr} \cdot \mathbf{n}}{\|\mathbf{y}_r\|^2}. \quad (13d)$$

The boundary condition $\boldsymbol{\sigma}\mathbf{n} = \gamma\kappa(\mathbf{x})\mathbf{n}$ consist of the scalar equations $\mathbf{n}^T\boldsymbol{\sigma}\mathbf{n} = \gamma\kappa(\mathbf{x})$ and $\mathbf{t}^T\boldsymbol{\sigma}\mathbf{n} = 0$ which are (with summation convection)

$$-p + n_i\mu(\partial_{x_j}u_i + \partial_{x_i}u_j)n_j = \gamma\kappa(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega, \quad t > 0, \quad (13e)$$

$$\mu t_i(\partial_{x_j}u_i + \partial_{x_i}u_j)n_j = 0, \quad \mathbf{x} \in \partial\Omega, \quad t > 0. \quad (13f)$$

The boundary condition for the pressure on a traction boundary comes from the normal component of (13a),

$$\mathbf{n}^T\boldsymbol{\sigma}\mathbf{n} = \gamma\kappa(\mathbf{x}). \quad (14)$$

In two dimensions

$$\mathbf{n}^T\boldsymbol{\sigma}\mathbf{n} = -p + 2\mu(u_x n_1^2 + v_y n_2^2 + (u_y + v_x)n_1 n_2). \quad (15)$$

The boundary conditions used for the velocity on a traction boundary are $\nabla \cdot \mathbf{u} = 0$ and $\mathbf{t}^T\boldsymbol{\tau}\mathbf{n} = 0$. These can be combined as

$$(\nabla \cdot \mathbf{u})\mathbf{n} + \mathbf{t}(\mathbf{t}^T\boldsymbol{\tau}\mathbf{n}) = 0. \quad (16)$$

From $I = \mathbf{n}\mathbf{n}^T + \mathbf{t}\mathbf{t}^T$, it follows that $\mathbf{t}\mathbf{t}^T = I - \mathbf{n}\mathbf{n}^T$ and thus (16) can be written as

$$(\nabla \cdot \mathbf{u})\mathbf{n} + (I - \mathbf{n}\mathbf{n}^T)\boldsymbol{\tau}\mathbf{n} = 0. \quad (17)$$

The form in (17) is convenient since it does not require forming tangent vectors. In addition, it is a good form to use when constructing the implicit time-stepping matrix to ensure that the matrix has non-zero diagonal entries.

3. Numerical approach

3.1. Discrete approximations in space

The discrete approach is based on the methods described in [1–3].

The method-of-lines approximation keeps time continuous and discretizes in space. Let $\mathbf{U}_j(t) \approx \mathbf{u}(\mathbf{x}_j, t)$, and $P_j(t) = p(\mathbf{x}_j, t)$. The discretization of the interior equations takes the form

$$\frac{d}{dt}\mathbf{U}_j + (\mathbf{U}_j \cdot \nabla_h)\mathbf{U}_j + \frac{1}{\rho}\nabla_h P_j = \nu\Delta_h\mathbf{U}_j - \mathcal{D}_h^{(2)}\mathbf{U}_j, \quad \mathbf{j} \in \Omega_h, \quad (18a)$$

$$\Delta_h P_j = \nabla_h \mathbf{U}_j : \nabla_h \mathbf{U}_j + \frac{c_d}{\Delta t} \nabla_h \cdot \mathbf{U}_j, \quad \mathbf{j} \in \Omega_h, \quad (18b)$$

where ∇_h and Δ_h denote discrete approximations to ∇ and Δ , respectively. The second-order artificial dissipation operator $\mathcal{D}_h^{(2)}$ is

$$\mathcal{D}_h^{(2)}\mathbf{U}_j \stackrel{\text{def}}{=} (a_1 + a_2\|\nabla_h \mathbf{U}_j\|_1) \left((-\Delta x^2 D_{+x} D_{-x}) + (-\Delta y^2 D_{+y} D_{-y}) \right), \quad (18c)$$

$$\|\nabla_h \mathbf{U}_j\|_1 = |D_{0x} V_{1,j}| + |D_{0x} V_{2,j}| + |D_{0y} V_{1,j}| + |D_{0y} V_{2,j}|, \quad (18d)$$

while c_d is the coefficient of the divergence damping.

3.2. Time-stepping algorithms

Time-stepping algorithms

AB2 A second-order accurate Adams-Bashforth scheme

PC2 A second-order accurate predictor-corrector scheme

IM2 A second-order accurate Implicit-explicit (IMEX) scheme.

3.3. Free surface evolution

Let $\boldsymbol{\eta}(s, t)$ denote the x-y coordinates of the free surface as a function of s and t . Here s is some parameterization of the surface. The free surface satisfies

$$\partial_t \boldsymbol{\eta} = \mathbf{u}(\boldsymbol{\eta}(s, t), t). \quad (19)$$

Here is a predictor corrector time-stepping algorithm to update the free surface from time t^n to t^{n+1} . The predictor step is a second-order accurate Adams-Bashforth scheme to compute $\boldsymbol{\eta}_i^p \approx \boldsymbol{\eta}^{n+1}$,

$$\boldsymbol{\eta}_i^p = \boldsymbol{\eta}_i^n + \Delta t \left(\frac{3}{2} \mathbf{U}_i^n - \frac{1}{2} \mathbf{U}_i^{n-1} \right). \quad (20)$$

After the fluid variables have been updated to t^{n+1} , the free-surface positions are corrected with a second-order accurate Adams-Moulton scheme,

$$\boldsymbol{\eta}_i^{n+1} = \boldsymbol{\eta}_i^n + \Delta t \left(\frac{1}{2} \mathbf{U}_i^{n+1} + \frac{1}{2} \mathbf{U}_i^n \right). \quad (21)$$

4. Numerical Results

4.1. Manufactured solutions

The Matlab code `runGridConvergence.m` can be used to run grid convergence tests and create a LaTeX table. Here is an example, (for the implicit method we specify the Δt to use on the coarsest grid, and this is halved each resolution)

```
runGridConvergence -ts=ab2 -tf=.25 -ms=trig -numResolutions=4 runGridConvergence -ts=pc2 -tf=.25 -ms=trig
-numResolutions=4 runGridConvergence -ts=im2 -tf=.25 -ms=trig -numResolutions=3 -dtMax=.01
```

INS: ts=ab2, BC=nnnn, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$ \nabla \cdot \mathbf{u} $	r
10	1.09e-02	5.57e-02		4.18e-03		2.76e-03		2.54e-02	
20	2.81e-03	1.47e-02	3.8	4.72e-04	8.8	2.96e-04	9.3	2.01e-03	12.6
40	7.02e-04	3.29e-03	4.5	1.17e-04	4.0	6.56e-05	4.5	1.40e-04	14.3
80	1.76e-04	9.19e-04	3.6	2.92e-05	4.0	1.56e-05	4.2	1.11e-05	12.7

INS: ts=pc2, BC=nnnn, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$ \nabla \cdot \mathbf{u} $	r
10	1.92e-02	4.46e-02		6.31e-03		4.17e-03		4.00e-02	
20	4.90e-03	1.16e-02	3.8	4.71e-04	13.4	3.54e-04	11.8	3.28e-03	12.2
40	1.23e-03	2.92e-03	4.0	1.17e-04	4.0	6.59e-05	5.4	2.31e-04	14.2
80	3.08e-04	7.41e-04	3.9	2.92e-05	4.0	1.57e-05	4.2	1.79e-05	12.9

INS: ts=im2, BC=nnnn, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$ \nabla \cdot \mathbf{u} $	r
10	1.00e-02	4.54e-02		3.89e-03		2.89e-03		2.32e-02	
20	5.00e-03	1.16e-02	3.9	4.71e-04	8.3	3.56e-04	8.1	3.34e-03	7.0
40	2.50e-03	2.90e-03	4.0	1.16e-04	4.0	6.89e-05	5.2	4.33e-04	7.7
80	1.25e-03	7.36e-04	3.9	2.91e-05	4.0	1.60e-05	4.3	5.83e-05	7.4

4.2. Taylor Green vortex

The Taylor Green vortex is an exact solution given by

$$u = +\alpha \sin(kx) \cos(ky) F(t) \quad (22a)$$

$$v = -\alpha \cos(kx) \sin(ky) F(t) \quad (22b)$$

$$p = \frac{\rho\alpha^2}{4} (\cos(2kx) + \cos(2ky)) F^2(t) \quad (22c)$$

$$F(t) \stackrel{\text{def}}{=} \exp(-2\nu k^2 t) \quad (22d)$$

where α is a parameter defining the initial amplitude.

Figure 1 shows some results using the command

```
ins -ts=im2 -tf=.5 -ms=none -knownSolution=TaylorGreen -idebug=1 -nu=0.1 -bcs=nnnn -N0=100 -plotOption=1
```

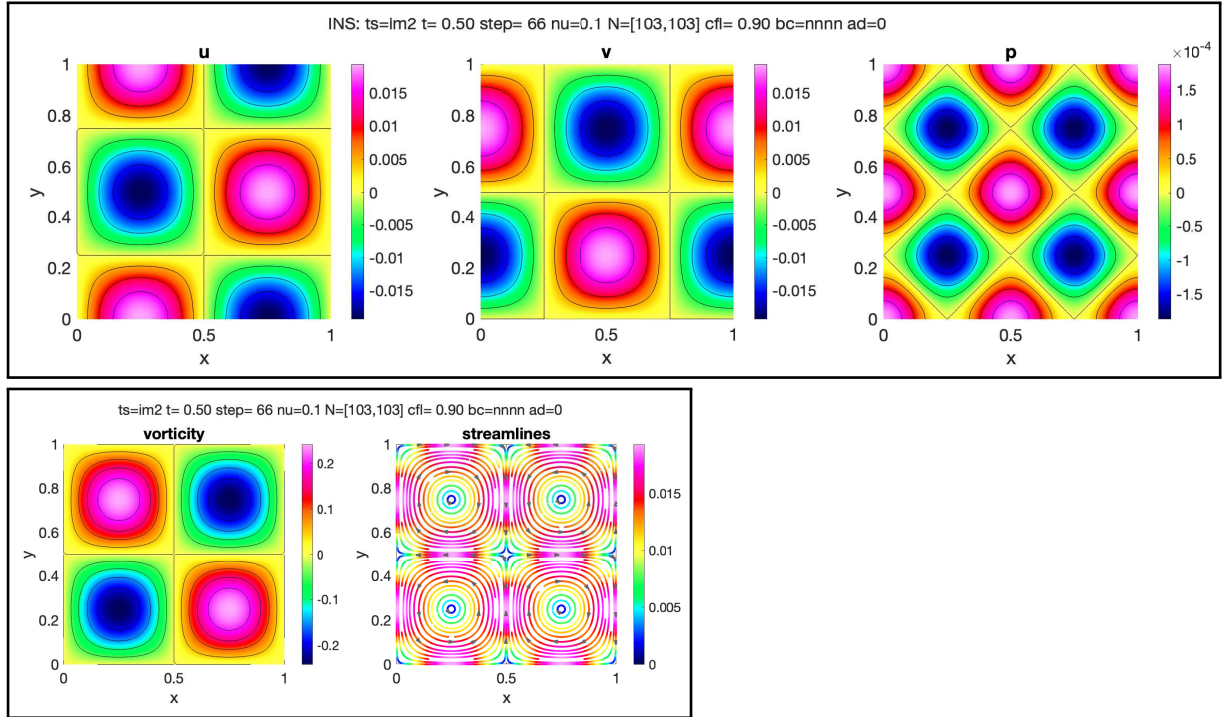


Figure 1: Taylor Green vortex

4.3. Shear flow in a periodic channel

Figure 2 shows the roll-up of a shear flow in periodic channel. The initial condition is perturbed shear flow

$$u(x, y, 0) = \tanh(\beta(y - y_m)), \quad (23)$$

$$v(x, y, 0) = \delta \cos(2\pi(y - y_m)) \sin(2\pi k_x x) \quad (24)$$

```
ins -ts=pc2 -tf=2.0 -tp=0.5 -ms=none -knownSolution=none -idebug=1 -nu=.0002 -ya=-.5 -yb=1.5 -xb=2
-bcs=ppnn -N0=800 -ic=shear -movieMode=1 -plotOption=3 -plotErrors=0 -kx=1 -checkTimeStep=20
-ad=1 -ad21=.1 -ad22=.1 -cfl=0.95
```

Artificial dissipation is added.

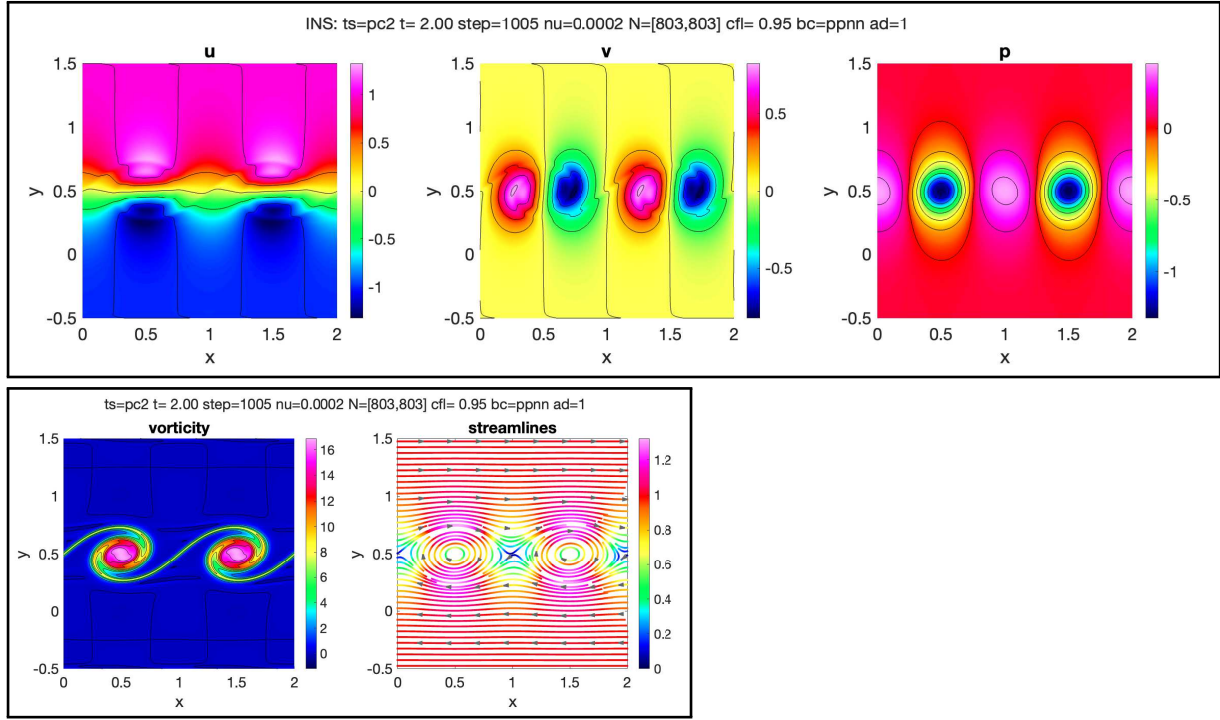


Figure 2: Rollup of a shear flow in a channel.

Here are the timings on a Mac-book pro showing that most of the time is spent in solving for the pressure.

```
----- TIMINGS pc2 Nx=800 Ny=800 Nt= 1005 -----
              cpu (s)    %
Total          5.89e+02  100.0
  setup         7.09e+01   12.0
  getUt         1.16e+02   19.7
implicit solve  0.00e+00    0.0
implicit factor 0.00e+00    0.0
pressure solve  3.75e+02   63.7
    bc          3.50e-01    0.1
plotting       2.03e+01    3.4
```

References

- [1] G. S. Chesshire, W. D. Henshaw, Composite overlapping meshes for the solution of partial differential equations 90 (1) (1990) 1–64, [publications/cgns1990.pdf](#).
- [2] W. D. Henshaw, N. A. Petersson, A split-step scheme for the incompressible Navier-Stokes equations, in: M. M. Hafez (Ed.), Numerical Simulation of Incompressible Flows, World Scientific, 2003, pp. 108–125, [publications/henshawPetersson2001.pdf](#).
- [3] F. Meng, J. W. Banks, W. D. Henshaw, D. W. Schwendeman, Fourth-order accurate fractional-step IMEX schemes for the incompressible Navier-Stokes equations on moving overlapping grids, Computer Methods in Applied Mechanics and Engineering 366 (2020) 113040, [publications/FourthOrderAccurateFractionalStepIMEXSchemeForINSOnMovingGrids_MengBanksHenshawSchwendeman2020.pdf](#).